PYTHON IN MATHEMATICS

What is Python Implementation in Mathematics?

Python implementation in mathematics means using the Python programming language to solve mathematical problems, perform calculations, analyze data, visualize graphs, and model mathematical concepts efficiently.

Instead of solving complex equations manually, Python allows students and researchers to compute results accurately, explore patterns, and visualize mathematical relationships.

Python is widely used in:

- Pure mathematics
- Applied mathematics
- Engineering mathematics
- Statistics & data science
- AI & machine learning

Why Python is Used in Mathematics

Python helps to:

- Solve equations
- Perform algebraic operations
- Work with matrices and vectors
- Plot graphs
- Perform statistical analysis
- Implement numerical methods

Popular Python libraries:

- **Math** – basic mathematical operations
- **NumPy** – numerical computing
- **SymPy** – symbolic mathematics
- **Matplotlib** – graph plotting

Program 1 — Quadratic Equation Solver

```
import math

a = float(input("a: "))
b = float(input("b: "))
c = float(input("c: "))

d = b*b - 4*a*c

x1 = (-b + math.sqrt(d))/(2*a)
x2 = (-b - math.sqrt(d))/(2*a)

print("Roots:", x1, x2)
```

Program 2 — Factorial Calculation

```
n = int(input("Enter Number: "))

fact = 1
for i in range(1,n+1):
    fact *= i

print("Factorial:", fact)
```

Program 3 — Mean & Standard Deviation

```
import statistics

data = [10,20,30,40]

print("Mean:", statistics.mean(data))
print("Std Dev:", statistics.stdev(data))
```

Program 4 — Linear Equation Solver

```
a = float(input("a: "))
b = float(input("b: "))

x = -b/a
print("Solution:", x)
```

Program 5 — Simple Graph Plot

```
import matplotlib.pyplot as plt

x = [1,2,3,4]
y = [2,4,6,8]

plt.plot(x,y)
plt.show()
```

Program 6 — Symbolic Differentiation

```
import sympy as sp

x = sp.symbols('x')
expr = x**3 + 2*x

print("Derivative:", sp.diff(expr,x))
```

Program 7 — Definite Integration

```
import sympy as sp

x = sp.symbols('x')

result = sp.integrate(x**2,(x,0,5))
print("Integration:", result)
```

Program 8 — Matrix Multiplication

```python
import numpy as np

A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

print("Result:", np.dot(A,B))
```

Program 9 — Eigenvalues Calculation

```python
import numpy as np

A = np.array([[2,1],[1,2]])

values, vectors = np.linalg.eig(A)

print("Eigenvalues:", values)
```

Program 10 — Numerical Root Finding

```python
from scipy.optimize import fsolve

def f(x):
    return x**2 - 4

root = fsolve(f,2)

print("Root:", root)
```

Program 11 — Taylor Series Expansion

```python
import sympy as sp

x = sp.symbols('x')
expr = sp.sin(x)

print(sp.series(expr,x,0,6))
```

Program 12 — Determinant of Matrix

```python
import numpy as np

A = np.array([[1,2,3],[4,5,6],[7,8,9]])

print("Determinant:", np.linalg.det(A))
```

Program 13 — Numerical Differentiation

```python
def f(x):
    return x**2

x = 3
h = 0.0001

derivative = (f(x+h)-f(x))/h
print("Derivative:", derivative)
```

Program 14 — Polynomial Roots

```python
import numpy as np

coeff = [1,-3,2]
roots = np.roots(coeff)

print("Roots:", roots)
```

Program 15 — Random Probability Simulation

```python
import random

success = 0

for i in range(100):
    if random.random() < 0.5:
        success +=1

print("Probability:", success/100)
```

Program 16 — Power Method (Eigenvalue)

```python
import numpy as np

A = np.array([[2,1],[1,2]])
x = np.array([1,1])

for i in range(5):
    x = np.dot(A,x)
    x = x/max(x)

print("Eigenvector:",x)
```

Program 17 — Numerical Second Derivative

```python
def f(x):
    return x**3

x = 2
h = 0.001

second = (f(x+h)-2*f(x)+f(x-h))/(h*h)
print("Second Derivative:",second)
```

Program 18 — LU Matrix Decomposition

```python
import scipy.linalg as la
import numpy as np

A = np.array([[4,3],[6,3]])
P,L,U = la.lu(A)

print("L:",L)
print("U:",U)
```

Program 19 — Markov Chain Simulation

```python
import numpy as np

P = np.array([[0.7,0.3],[0.4,0.6]])
state = np.array([1,0])

for i in range(5):
    state = np.dot(state,P)
    print(state)
```

Program 20 — Fourier Transform (Signal Analysis)

```python
import numpy as np

signal = np.sin(np.linspace(0,10,100))
freq = np.fft.fft(signal)

print(freq)
```

Program 21 — Gradient Descent Optimization

```python
x = 5
lr = 0.1

for i in range(10):
    grad = 2*x
    x = x - lr*grad

print("Minimum near:",x)
```

Program 22 — Monte Carlo π Estimation

```python
import random

inside = 0
total = 1000

for i in range(total):
    x = random.random()
    y = random.random()
    if x*x+y*y <=1:
        inside+=1

print("Pi ≈",4*inside/total)
```

Program 23 — Simpson's Rule Integration

```python
def f(x):
    return x**2

a = 0
b = 4

result = (b-a)/6*(f(a)+4*f((a+b)/2)+f(b))

print("Integration:",result)
```

Program 24 — Newton–Raphson Method (Root Finding)

```python
def f(x):
    return x**3 - x - 2

def df(x):
    return 3*x**2 - 1

x = 1

for i in range(5):
    x = x - f(x)/df(x)

print("Root:",x)
```

Program 25 — Euler Method (Differential Equation Solver)

```python
def f(x,y):
    return x+y

x = 0
y = 1
h = 0.1

for i in range(5):
    y = y + h*f(x,y)
    x = x + h
    print("x=",x," y=",y)
```

Mini Project

Title: Machine Learning Based Differential Equation Solver using Python

Problem Statement

Solving complex differential equations:

- Takes long computation time
- Needs numerical approximation
- Difficult for non-linear systems

Need a Python-based intelligent solver that:

- Learns mathematical patterns
- Predicts solutions automatically
- Visualizes results

```python
import streamlit as st
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

st.title("AI Based Differential Equation Solver (PG Project)")

# ---------------- USER INPUT ----------------
equation_type = st.selectbox(
    "Select Equation Model",
    ["Simple Differential", "Physics Motion Model", "Biological Growth Model"]
)

epochs = st.slider("Training Epochs",50,500,200)
```

```python
# ---------------- DATA GENERATION ----------------
x = np.linspace(0,1,100)

if equation_type == "Simple Differential":
    y_true = np.exp(x) - x - 1

elif equation_type == "Physics Motion Model":
    y_true = 5*x**2 + 2*x

elif equation_type == "Biological Growth Model":
    y_true = 1/(1+np.exp(-10*(x-0.5)))

X = x.reshape(-1,1)
Y = y_true.reshape(-1,1)

# ---------------- NUMERICAL METHOD ----------------
y_numerical = np.cumsum(x)/len(x)
```

```python
# ---------------- ML MODEL ----------------
model = tf.keras.Sequential([
    tf.keras.layers.Dense(20,activation='relu',input_shape=(1,)),
    tf.keras.layers.Dense(20,activation='relu'),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer='adam',loss='mse')
model.fit(X,Y,epochs=epochs,verbose=0)

pred = model.predict(X)

# ---------------- ERROR CALCULATION ----------------
error = np.mean(np.abs(pred.flatten()-y_true))
st.write("Average Prediction Error:",error)
```

```python
# ---------------- GRAPH ----------------
fig = plt.figure()
plt.plot(x,y_true,label="True Solution")
plt.plot(x,pred,label="ML Prediction")
plt.plot(x,y_numerical,label="Numerical Approx")
plt.legend()

st.pyplot(fig)
```